LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5

CONNECT TO THE INTERNET

Oleh:

Nazmi Hakim NIM. 2310817210012

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nazmi Hakim
NIM : 2310817210012

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Raka Azwar NIM. 2210817210012	Andreyan Rizky Baskara, S.Kom., M.Kom. NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 6
B. Output Program 35
C. Pembahasan 39
D. Tautan Git 53

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Jetpack Compose 35
Gambar 2. Screenshot Hasil Jawaban Soal 1 Jetpack Compose 36
Gambar 3. Screenshot Hasil Jawaban Soal 1 Jetpack Compose 37
Gambar 4. Screenshot Hasil Jawaban Soal 1 Jetpack Compose 38
Gambar 5. Screenshot Hasil Jawaban Soal 1 Jetpack Compose 39

DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1 Jetpack Compose.....6
Tabel 2. Source Code Jawaban Soal 1 Jetpack Compose.....8
Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose.....9
Tabel 4. Source Code Jawaban Soal 1 Jetpack Compose.....10
Tabel 5. Source Code Jawaban Soal 1 Jetpack Compose.....11
Tabel 6. Source Code Jawaban Soal 1 Jetpack Compose.....12
Tabel 7. Source Code Jawaban Soal 1 Jetpack Compose.....14
Tabel 8. Source Code Jawaban Soal 1 Jetpack Compose.....15
Tabel 9. Source Code Jawaban Soal 1 Jetpack Compose.....16
Tabel 10. Source Code Jawaban Soal 1 Jetpack Compose.....18
Tabel 11. Source Code Jawaban Soal 1 Jetpack Compose.....21
Tabel 12. Source Code Jawaban Soal 1 Jetpack Compose.....26
Tabel 13. Source Code Jawaban Soal 1 Jetpack Compose.....30
Tabel 14. Source Code Jawaban Soal 1 Jetpack Compose.....31
Tabel 15. Source Code Jawaban Soal 1 Jetpack Compose.....34

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
b. Gunakan KotlinX Serialization sebagai library JSON.
c. Gunakan library seperti Coil atau Glide untuk image loading.
d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: https://developer.themoviedb.org/docs/getting-started
e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

MainActivity.kt :

Tabel 1. Source Code Jawaban Soal 1 Jetpack Compose

```
1  package com.example.listcompose
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.compose.setContent
6  import androidx.lifecycle.viewmodel.compose.viewModel
7  import androidx.navigation.NavType
8  import androidx.navigation.compose.*
9  import androidx.navigation.navArgument
10 import com.example.listcompose.data.local.AppDatabase
11 import
12 com.example.listcompose.data.preferences.PreferencesManager
13 import com.example.listcompose.data.remote.MovieRepository
14 import com.example.listcompose.data.remote.RetrofitInstance
15 import com.example.listcompose.ui.screens.*
16 import com.example.listcompose.ui.theme.ListComposeTheme
17 import com.example.listcompose.viewmodel.MovieViewModel
18 import com.example.listcompose.viewmodel.MovieViewModelFactory
19 import timber.log.Timber
```

```
20
21 class MainActivity : ComponentActivity() {
22     override fun onCreate(savedInstanceState: Bundle?) {
23         super.onCreate(savedInstanceState)
24         Timber.plant(Timber.DebugTree())
25
26         val context = this
27         val database = AppDatabase.getInstance(context)
28         val repository = MovieRepository(
29             apiService = RetrofitInstance.api,
30             movieDao = database.movieDao()
31         )
32         val preferencesManager = PreferencesManager(context)
33         val viewModelFactory =
34 MovieViewModelFactory(application, repository,
35 preferencesManager)
36
37         setContent {
38             ListComposeTheme {
39                 val navController = rememberNavController()
40                 val movieViewModel: MovieViewModel =
41 viewModel(factory = viewModelFactory)
42                 val username = movieViewModel.getUsername()
43
44                 NavHost(navController, startDestination =
45 "home") {
46                     composable("home") {
47                         HomeScreen(
48                             username = username,
49                             onNavigateToMovies = {
50                                 movieViewModel.loadMovies()
51
52 navController.navigate("movies")
53                             },
54                             onNavigateToLocalMovies = {
55
56 navController.navigate("local_movies")
57                             },
58                             onNavigateToSettings = {
59
60 navController.navigate("settings")
61                             },
62                             navController = navController,
63                             viewModel = movieViewModel
64                         )
65                     }
66
67                     composable("movies") {
68                         MovieListScreen(
69                             navController = navController,
70                             viewModel = movieViewModel,
71                             isLocal = false
```

```
                        )
                    }

                    composable("local_movies") {
                        MovieListScreen(
                            navController = navController,
                            viewModel = movieViewModel,
                            isLocal = true
                        )
                    }

                    composable(
                        "movie_detail/{id}",
                        arguments = listOf(navArgument("id") {
type = NavType.IntType })
                    ) { backStackEntry ->
                        val id =
backStackEntry.arguments?.getInt("id") ?: 0
                        MovieDetailScreen(
                            movieId = id,
                            viewModel = movieViewModel
                        )
                    }

                    composable("settings") {
                        SettingsScreen(
                            viewModel = movieViewModel,
                            onBackClick = {
navController.popBackStack() }
                        )
                    }
                }
            }
        }
    }
}
```

data :

local :

AppDatabase.kt :

Tabel 2. Source Code Jawaban Soal 1 Jetpack Compose

```
package com.example.listcompose.data.local

import android.content.Context
import androidx.room.Database
import androidx.room.Room
```

```
6  import androidx.room.RoomDatabase
7  import com.example.listcompose.data.model.Movie
8
9  @Database(
10     entities = [Movie::class],
11     version = 7,
12     exportSchema = true
13 )
14 abstract class AppDatabase : RoomDatabase() {
15     abstract fun movieDao(): MovieDao
16
17     companion object {
18         fun getInstance(context: Context): AppDatabase {
19             return Room.databaseBuilder(
20                 context,
21                 AppDatabase::class.java,
22                 "movie_db"
23             )
24                 .fallbackToDestructiveMigration()
25                 .build()
26         }
27     }
28 }
```

MovieDao.kt :

Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose

```
1  package com.example.listcompose.data.local
2
3  import androidx.room.Dao
4  import androidx.room.Insert
5  import androidx.room.OnConflictStrategy
6  import androidx.room.Query
7  import com.example.listcompose.data.model.Movie
8  import kotlinx.coroutines.flow.Flow
9
10 @Dao
11 interface MovieDao {
12     @Query("SELECT * FROM movies")
13     fun getAllMovies(): Flow<List<Movie>>
14
15     @Query("SELECT MAX(last_updated) FROM movies")
16     suspend fun getLastUpdateTime(): Long?
17
18     @Insert(onConflict = OnConflictStrategy.REPLACE)
19     suspend fun insertMovies(movies: List<Movie>)
20
21     @Query("DELETE FROM movies")
```

```
22      suspend fun clearAll()
23
24      @Query("SELECT * FROM movies WHERE is_local = 1")
25      fun getLocalMovies(): Flow<List<Movie>>
26
27      @Query("SELECT * FROM movies WHERE id = :id")
28      suspend fun getMovieById(id: Int): Movie?
29
30  }
```

model :

Movie.kt :

Tabel 4. Source Code Jawaban Soal 1 Jetpack Compose

```
1   package com.example.listcompose.data.model
2
3   import androidx.room.ColumnInfo
4   import androidx.room.Entity
5   import androidx.room.PrimaryKey
6   import kotlinx.serialization.SerialName
7   import kotlinx.serialization.Serializable
8
9   @Serializable
10  @Entity(tableName = "movies")
11  data class Movie(
12      @PrimaryKey
13      val id: Int,
14
15      val title: String,
16      val overview: String,
17
18      @SerialName("poster_path")
19      @ColumnInfo(name = "poster_path")
20      val posterPath: String?,
21
22      @SerialName("vote_average")
23      @ColumnInfo(name = "vote_average")
24      val voteAverage: Double,
25
26      @SerialName("release_date")
27      @ColumnInfo(name = "release_date")
28      val releaseDate: String,
29
30      val popularity: Double,
31
32      // Additional fields for local storage
33      @ColumnInfo(name = "is_local")
```

```
    val isLocal: Boolean = false,

    @ColumnInfo(name = "last_updated")
    val lastUpdated: Long = System.currentTimeMillis(),

    @ColumnInfo(name = "tmdb_url")
    val tmdbUrl: String = "",

    // Optional fields from API that we might not always use
    @SerialName("adult")
    val adult: Boolean? = null,

    @SerialName("backdrop_path")
    val backdropPath: String? = null
)

@Serializable
data class MovieResponse(
    val page: Int,
    val results: List<Movie>,
    @SerialName("total_pages") val totalPages: Int,
    @SerialName("total_results") val totalResults: Int
)
```

preferences :

PreferencesManager.kt :

Tabel 5. Source Code Jawaban Soal 1 Jetpack Compose

```
package com.example.listcompose.data.preferences

import android.content.Context
import android.content.SharedPreferences
import androidx.core.content.edit

class PreferencesManager(context: Context) {
    private val prefs: SharedPreferences =
context.getSharedPreferences("user_prefs",
Context.MODE_PRIVATE)

    companion object {
        private const val KEY_USERNAME = "username"
        private const val KEY_LAST_VIEWED_MOVIE_ID =
"last_viewed_movie_id"
    }

    fun saveUsername(username: String) {
        prefs.edit {
            putString(KEY_USERNAME, username)
```

```
        }
    }

    fun getUsername(): String {
        return prefs.getString(KEY_USERNAME, "") ?: ""
    }

    fun saveLastViewedMovieId(movieId: Int) {
        prefs.edit { putInt(KEY_LAST_VIEWED_MOVIE_ID, movieId)
}
    }

    fun getLastViewedMovieId(): Int {
        return prefs.getInt(KEY_LAST_VIEWED_MOVIE_ID, -1)
    }
}
```

remote :

MovieRepository.kt :

Tabel 6. Source Code Jawaban Soal 1 Jetpack Compose

```
package com.example.listcompose.data.remote

import com.example.listcompose.BuildConfig
import com.example.listcompose.data.local.MovieDao
import com.example.listcompose.data.model.Movie
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.catch
import kotlinx.coroutines.flow.first
import timber.log.Timber

class MovieRepository(
    private val apiService: TmdbApiService,
    private val movieDao: MovieDao
) {
    companion object {
        private const val CACHE_EXPIRATION_TIME_MS = 30 * 60 *
1000L
    }

    private var currentPage = 1
    private var totalPages = 1

    suspend fun refreshMovies(forceRefresh: Boolean = false):
Boolean {
        return try {
            val cachedMovies = movieDao.getAllMovies().first()
            val shouldRefresh = forceRefresh ||
```

```
28  isCacheExpired() || cachedMovies.isEmpty()
29
30              if (!shouldRefresh && currentPage >= totalPages) {
31                  return false
32              }
33
34              if (shouldRefresh) {
35                  currentPage = 1
36                  if (forceRefresh) {
37                      movieDao.clearAll()
38                  }
39              }
40
41              val response = apiService.getPopularMovies(
42                  apiKey = BuildConfig.TMDB_API_KEY,
43                  language = "en-US",
44                  page = currentPage
45              )
46
47              totalPages = response.totalPages
48
49              response.results.takeIf { it.isNotEmpty() }?.let {
50  movies ->
51                  val entities = movies.map { movie ->
52                      Movie(
53                          id = movie.id,
54                          title = movie.title,
55                          overview = movie.overview,
56                          posterPath = movie.posterPath ?: "",
57                          voteAverage = movie.voteAverage,
58                          releaseDate = movie.releaseDate,
59                          popularity = movie.popularity,
60                          lastUpdated =
61  System.currentTimeMillis(),
62                          tmdbUrl =
63  "https://www.themoviedb.org/movie/${movie.id}"
64                      )
65                  }
66                  movieDao.insertMovies(entities)
67
68                  if (!forceRefresh && currentPage < totalPages)
69  {
70                      currentPage++
71                  }
72              }
73              true
74          } catch (e: Exception) {
75              Timber.e(e, "Failed to refresh movies")
76              throw e
77          }
78      }
79
```

```
    private suspend fun isCacheExpired(): Boolean {
        return movieDao.getLastUpdateTime()?.let { lastUpdated
->
            System.currentTimeMillis() - lastUpdated >
CACHE_EXPIRATION_TIME_MS
        } ?: true
    }

    fun getMovies(): Flow<List<Movie>> =
movieDao.getAllMovies()
        .catch { e ->
            Timber.e(e, "Error loading movies from DB")
            emit(emptyList())
        }

    fun getLocalMovies(): Flow<List<Movie>> =
movieDao.getLocalMovies()

    suspend fun saveMovieToLocal(movie: Movie) {
        movieDao.insertMovies(listOf(movie.copy(
            lastUpdated = System.currentTimeMillis(),
            isLocal = true
        )))
}
    suspend fun getMovieById(id: Int): Movie? {
        return movieDao.getMovieById(id)
    }
}
```

RetrofitInstance.kt :

Tabel 7. Source Code Jawaban Soal 1 Jetpack Compose

```
package com.example.listcompose.data.remote

import com.jakewharton.retrofit2.converter.kotlinx
.serialization.asConverterFactory
import kotlinx.serialization.json.Json
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Retrofit

object RetrofitInstance {
    private const val BASE_URL =
"https://api.themoviedb.org/3/"

    private val json = Json {
        ignoreUnknownKeys = true
```

```
        isLenient = true
        coerceInputValues = true
    }

    val api: TmdbApiService by lazy {
        val loggingInterceptor = HttpLoggingInterceptor().apply
{
            level = HttpLoggingInterceptor.Level.BODY
        }

        val client = OkHttpClient.Builder()
            .addInterceptor(loggingInterceptor)
            .build()

        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .client(client)
            .addConverterFactory(json.asConverterFactory
("application/json".toMediaType()))
            .build()
            .create(TmdbApiService::class.java)
    }
}
```

TmdbApiService :

Tabel 8. Source Code Jawaban Soal 1 Jetpack Compose

```
package com.example.listcompose.data.remote

import com.example.listcompose.data.model.MovieResponse
import retrofit2.http.GET
import retrofit2.http.Query

interface TmdbApiService {
    @GET("movie/popular")
    suspend fun getPopularMovies(
        @Query("api_key") apiKey: String,
        @Query("language") language: String = "en-US",
        @Query("page") page: Int = 1
    ): MovieResponse
}
```

ui :

components :

Card.kt :

Tabel 9. Source Code Jawaban Soal 1 Jetpack Compose

```
1   package com.example.listcompose.ui.components
2
3   import androidx.compose.foundation.Image
4   import androidx.compose.foundation.layout.*
5   import androidx.compose.foundation.shape.RoundedCornerShape
6   import androidx.compose.material.icons.Icons
7   import
8   androidx.compose.material.icons.automirrored.filled.TrendingUp
9   import androidx.compose.material.icons.filled.Star
10  import androidx.compose.material3.*
11  import androidx.compose.runtime.Composable
12  import androidx.compose.ui.Alignment
13  import androidx.compose.ui.Modifier
14  import androidx.compose.ui.draw.clip
15  import androidx.compose.ui.graphics.Color
16  import androidx.compose.ui.layout.ContentScale
17  import androidx.compose.ui.res.stringResource
18  import androidx.compose.ui.unit.dp
19  import coil.compose.rememberAsyncImagePainter
20  import com.example.listcompose.R
21  import com.example.listcompose.data.model.Movie
22
23  @Composable
24  fun MovieCard(
25      movie: Movie,
26      onDetailClick: () -> Unit,
27      onSaveClick: (() -> Unit)? = null
28  ) {
29      Card(
30          modifier = Modifier
31              .padding(8.dp)
32              .fillMaxWidth(),
33          elevation =
34  CardDefaults.cardElevation(defaultElevation = 4.dp)
35      ) {
36          Column(modifier = Modifier.padding(16.dp)) {
37
38              Image(
39                  painter = rememberAsyncImagePainter(
40                      model =
41  "https://image.tmdb.org/t/p/w500${movie.posterPath}"
42                  ),
43                  contentDescription = movie.title,
```

```
                modifier = Modifier
                    .fillMaxWidth()
                    .height(200.dp)
                    .clip(RoundedCornerShape(8.dp)),
                contentScale = ContentScale.Crop
            )

            Spacer(modifier = Modifier.height(8.dp))

            Text(
                text = movie.title,
                style = MaterialTheme.typography.titleLarge
            )

            Row(
                modifier = Modifier.padding(top = 4.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Icon(
                    imageVector = Icons.Default.Star,
                    contentDescription =
stringResource(R.string.rating),
                    tint = Color(0xFFFFD700)
                )
                Text(
                    text = " ${movie.voteAverage}/10",
                    modifier = Modifier.padding(start = 4.dp)
                )

                Spacer(modifier = Modifier.width(8.dp))

                Icon(
                    imageVector =
Icons.AutoMirrored.Filled.TrendingUp,
                    contentDescription =
stringResource(R.string.popularity),
                    tint = Color(0xFF4CAF50)
                )
                Text(
                    text = " ${movie.popularity.toInt()}",
                    modifier = Modifier.padding(start = 4.dp)
                )

                Spacer(modifier = Modifier.width(8.dp))

                Text(text = movie.releaseDate)
            }

            Text(
                text = movie.overview.take(100) + "...",
                modifier = Modifier.padding(top = 8.dp),
                style = MaterialTheme.typography.bodyMedium
```

```
96              )
97
98              Button(
99                  onClick = onDetailClick,
100                 modifier = Modifier
101                     .fillMaxWidth()
102                     .padding(top = 16.dp)
103             ) {
104
105 Text(stringResource(R.string.view_details_button))
106             }
107
108             if (onSaveClick != null) {
109                 Button(
110                     onClick = onSaveClick,
111                     modifier = Modifier
112                         .fillMaxWidth()
113                         .padding(top = 8.dp)
                    ) {

    Text(stringResource(R.string.save_to_local_button))
                    }
                }
            }
        }
    }
```

screens :

HomeScreen.kt :

Tabel 10. Source Code Jawaban Soal 1 Jetpack Compose

```
1   package com.example.listcompose.ui.screens
2
3   import androidx.compose.foundation.layout.Arrangement
4   import androidx.compose.foundation.layout.Column
5   import androidx.compose.foundation.layout.Spacer
6   import androidx.compose.foundation.layout.fillMaxSize
7   import androidx.compose.foundation.layout.fillMaxWidth
8   import androidx.compose.foundation.layout.height
9   import androidx.compose.foundation.layout.padding
10  import androidx.compose.material3.Button
11  import androidx.compose.material3.ButtonDefaults
12  import androidx.compose.material3.MaterialTheme
13  import androidx.compose.material3.Text
14  import androidx.compose.runtime.Composable
15  import androidx.compose.runtime.collectAsState
16  import androidx.compose.runtime.derivedStateOf
```

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.example.listcompose.viewmodel.MovieViewModel

@Composable
fun HomeScreen(
    username: String,
    onNavigateToMovies: () -> Unit,
    onNavigateToLocalMovies: () -> Unit,
    onNavigateToSettings: () -> Unit,
    navController: NavController,
    viewModel: MovieViewModel = viewModel()
) {
    // Collect the local movies to check if last viewed movie
exists
    val localMovies by viewModel.localMovies.collectAsState()

    // Get the last viewed movie and check if it exists in
local movies
    val lastViewedMovie by remember {
        derivedStateOf {
            viewModel.getLastViewedMovie()?.takeIf { movie ->
                localMovies.any { it.id == movie.id }
            }
        }
    }

    Column(
        modifier = Modifier
            .padding(16.dp)
            .fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        if (username.isNotEmpty()) {
            Text(
                text = "Welcome, $username",
                style =
MaterialTheme.typography.headlineMedium,
                modifier = Modifier.padding(bottom = 16.dp))
        }

        Spacer(modifier = Modifier.height(32.dp))

        // Main buttons
        Column(
            modifier = Modifier
                .fillMaxWidth()
```

```
69                  .weight(1f),
70              verticalArrangement = Arrangement.Center
71          ) {
72              Button(
73                  onClick = onNavigateToMovies,
74                  modifier = Modifier
75                      .fillMaxWidth()
76                      .padding(vertical = 8.dp),
77                  colors = ButtonDefaults.buttonColors(
78                      containerColor =
79  MaterialTheme.colorScheme.primaryContainer,
80                      contentColor =
81  MaterialTheme.colorScheme.onPrimaryContainer
82                  )
83              ) {
84                  Text("Browse TMDB Movies")
85              }
86
87              Button(
88                  onClick = onNavigateToLocalMovies,
89                  modifier = Modifier
90                      .fillMaxWidth()
91                      .padding(vertical = 8.dp),
92                  colors = ButtonDefaults.buttonColors(
93                      containerColor =
94  MaterialTheme.colorScheme.primaryContainer,
95                      contentColor =
96  MaterialTheme.colorScheme.onSecondaryContainer
97                  )
98              ) {
99                  Text("View Saved Movies")
100             }
101         }
102
103         // Bottom section with last viewed and settings
104         Column(
105             modifier = Modifier.fillMaxWidth(),
106             verticalArrangement = Arrangement.Bottom
107         ) {
108             if (lastViewedMovie != null) {
109                 Text(
110                     text = "Last viewed saved movies:
111 ${lastViewedMovie!!.title}",
112                     style =
113 MaterialTheme.typography.bodyMedium,
114                     modifier = Modifier.padding(bottom = 8.dp)
115                 )
116                 Button(
117                     onClick = {
118
119 navController.navigate("movie_detail/${lastViewedMovie!!.id}")
120 {
```

```
                            // Clear back stack to avoid
multiple home screens

popUpTo(navController.graph.startDestinationId)
                            launchSingleTop = true
                        }
                    },
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(top = 8.dp),
                    colors = ButtonDefaults.buttonColors(
                        containerColor =
MaterialTheme.colorScheme.tertiaryContainer,
                        contentColor =
MaterialTheme.colorScheme.onTertiaryContainer
                    )
                ) {
                    Text("View Again")
                }
            }

            Button(
                onClick = onNavigateToSettings,
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(top = 8.dp),
                colors = ButtonDefaults.buttonColors(
                    containerColor =
MaterialTheme.colorScheme.secondaryContainer,
                    contentColor =
MaterialTheme.colorScheme.onSecondaryContainer
                )
            ) {
                Text("Settings")
            }
        }
    }
}
```

MovieDetailScreen.kt :

Tabel 11. Source Code Jawaban Soal 1 Jetpack Compose

```
1  package com.example.listcompose.ui.screens
2
3  import android.content.Intent
4  import android.widget.Toast
5  import androidx.compose.foundation.Image
6  import androidx.compose.foundation.layout.*
```

```
7   import androidx.compose.foundation.rememberScrollState
8   import androidx.compose.foundation.shape.RoundedCornerShape
9   import androidx.compose.foundation.verticalScroll
10  import androidx.compose.material.icons.Icons
11  import
12  androidx.compose.material.icons.automirrored.filled.OpenInNew
13  import androidx.compose.material.icons.filled.Star
14  import androidx.compose.material3.*
15  import androidx.compose.runtime.*
16  import androidx.compose.ui.Alignment
17  import androidx.compose.ui.Modifier
18  import androidx.compose.ui.draw.clip
19  import androidx.compose.ui.graphics.Color
20  import androidx.compose.ui.layout.ContentScale
21  import androidx.compose.ui.platform.LocalContext
22  import androidx.compose.ui.res.stringResource
23  import androidx.compose.ui.unit.dp
24  import androidx.core.net.toUri
25  import coil.compose.rememberAsyncImagePainter
26  import com.example.listcompose.R
27  import com.example.listcompose.data.model.Movie
28  import com.example.listcompose.viewmodel.MovieViewModel
29
30  @Composable
31  fun MovieDetailScreen(
32      movieId: Int,
33      viewModel: MovieViewModel
34  ) {
35      val remoteMovies by viewModel.movieList.collectAsState()
36      val localMovies by viewModel.localMovies.collectAsState()
37      val context = LocalContext.current
38
39      var movie by remember { mutableStateOf<Movie?>(null) }
40      var isLoading by remember { mutableStateOf(true) }
41      var error by remember { mutableStateOf<String?>(null) }
42
43      LaunchedEffect(movieId, remoteMovies, localMovies) {
44          isLoading = true
45          error = null
46
47          val localMovie = localMovies.find { it.id == movieId }
48          if (localMovie != null) {
49              movie = localMovie
50              viewModel.saveLastViewedMovieId(movieId)
51              isLoading = false
52              return@LaunchedEffect
53          }
54
55          val remoteMovie = remoteMovies.find { it.id == movieId
56  }
57          if (remoteMovie != null) {
58              movie = remoteMovie
```

```
59              isLoading = false
60              return@LaunchedEffect
61          }
62
63          var errorResId: Int? = null
64
65          try {
66              val dbMovie = viewModel.getMovieById(movieId)
67              if (dbMovie != null) {
68                  movie = dbMovie
69                  if (dbMovie.isLocal) {
70                      viewModel.saveLastViewedMovieId(movieId)
71                  }
72              } else {
73                  errorResId = R.string.movie_not_found
74              }
75          } catch (e: Exception) {
76              errorResId = R.string.error_loading_movie
77          }
78
79          if (errorResId != null) {
80              error = context.getString(errorResId)
81          }
82
83          isLoading = false
84      }
85
86      if (isLoading) {
87          Box(
88              modifier = Modifier.fillMaxSize(),
89              contentAlignment = Alignment.Center
90          ) {
91              CircularProgressIndicator()
92          }
93          return
94      }
95
96      if (error != null || movie == null) {
97          Box(
98              modifier = Modifier.fillMaxSize(),
99              contentAlignment = Alignment.Center
100         ) {
101             Text(error ?:
102 stringResource(R.string.movie_data_unavailable))
103         }
104         return
105     }
106
107     val scrollState = rememberScrollState()
108     val movieToShow = movie!!
109
110     Column(
```

```
111         modifier = Modifier
112             .fillMaxSize()
113             .verticalScroll(scrollState)
114             .padding(16.dp)
115     ) {
116         Image(
117             painter = rememberAsyncImagePainter(
118                 model =
119 "https://image.tmdb.org/t/p/w500${movieToShow.posterPath}"
120             ),
121             contentDescription = movieToShow.title,
122             modifier = Modifier
123                 .fillMaxWidth()
124                 .height(300.dp)
125                 .clip(RoundedCornerShape(12.dp)),
126             contentScale = ContentScale.Crop
127         )
128 
129         Spacer(modifier = Modifier.height(16.dp))
130 
131         Text(
132             text = movieToShow.title,
133             style = MaterialTheme.typography.headlineLarge
134         )
135 
136         Row(
137             modifier = Modifier.padding(top = 8.dp),
138             verticalAlignment = Alignment.CenterVertically
139         ) {
140             Icon(
141                 imageVector = Icons.Default.Star,
142                 contentDescription =
143 stringResource(R.string.rating),
144                 tint = Color(0xFFFFD700)
145             )
146             Text(
147                 text = " ${movieToShow.voteAverage}/10",
148                 style = MaterialTheme.typography.titleMedium,
149                 modifier = Modifier.padding(start = 4.dp)
150             )
151             Spacer(modifier = Modifier.width(16.dp))
152             Text(
153                 text = movieToShow.releaseDate,
154                 style = MaterialTheme.typography.titleMedium
155             )
156         }
157 
158         Text(
159             text = movieToShow.overview,
160             style = MaterialTheme.typography.bodyLarge,
161             modifier = Modifier.padding(top = 16.dp)
162         )
```

```
163
164         Spacer(modifier = Modifier.height(24.dp))
165
166         Button(
167             onClick = {
168                 val tmdbUrl =
169 "https://www.themoviedb.org/movie/$movieId"
170                 try {
171                     val intent = Intent(Intent.ACTION_VIEW,
172 tmdbUrl.toUri())
173                     context.startActivity(intent)
174                 } catch (e: Exception) {
175                     Toast.makeText(
176                         context,
177
178 context.getString(R.string.unable_to_open_browser),
179                         Toast.LENGTH_SHORT
180                     ).show()
181                 }
182             },
183             modifier = Modifier
184                 .fillMaxWidth()
185                 .padding(top = 8.dp),
186             colors = ButtonDefaults.buttonColors(
187                 containerColor =
188 MaterialTheme.colorScheme.primary,
189                 contentColor =
190 MaterialTheme.colorScheme.onPrimary
191             )
192         ) {
193             Icon(
                 imageVector =
    Icons.AutoMirrored.Filled.OpenInNew,
                 contentDescription = null,
                 modifier =
    Modifier.size(ButtonDefaults.IconSize)
             )
             Spacer(Modifier.width(ButtonDefaults.IconSpacing))
             Text(stringResource(R.string.view_on_tmdb))
         }
     }
    }
```

MovieListScreen.kt :

Tabel 12. Source Code Jawaban Soal 1 Jetpack Compose

```
1   package com.example.listcompose.ui.screens
2
3   import androidx.compose.foundation.layout.*
4   import androidx.compose.foundation.lazy.LazyColumn
5   import androidx.compose.foundation.lazy.LazyListState
6   import androidx.compose.foundation.lazy.items
7   import androidx.compose.foundation.lazy.rememberLazyListState
8   import androidx.compose.material3.*
9   import androidx.compose.runtime.*
10  import androidx.compose.ui.Alignment
11  import androidx.compose.ui.Modifier
12  import androidx.compose.ui.res.stringResource
13  import androidx.compose.ui.unit.dp
14  import androidx.navigation.NavController
15  import com.example.listcompose.R
16  import com.example.listcompose.data.model.Movie
17  import com.example.listcompose.ui.components.MovieCard
18  import com.example.listcompose.viewmodel.MovieViewModel
19
20  @Composable
21  fun MovieListScreen(
22      navController: NavController,
23      viewModel: MovieViewModel,
24      isLocal: Boolean
25  ) {
26      val movieState by viewModel.movieState.collectAsState()
27      val movies by if (isLocal) {
28          viewModel.localMovies.collectAsState(initial =
29  emptyList())
30      } else {
31          viewModel.movieList.collectAsState()
32      }
33      val scrollState = rememberLazyListState()
34      val username by remember { derivedStateOf {
35  viewModel.getUsername() } }
36
37      LaunchedEffect(scrollState) {
38          snapshotFlow { scrollState.layoutInfo.visibleItemsInfo
39  }
40              .collect { visibleItems ->
41                  if (visibleItems.isNotEmpty() &&
42                      visibleItems.last().index >= movies.size -
43  5 &&
44                      !isLocal &&
45                      movieState !is
46  MovieViewModel.MovieState.Loading
```

```
                ) {
                    viewModel.loadMovies()
                }
            }
    }

    Column(modifier = Modifier.fillMaxSize()) {
        if (!isLocal) {
            Button(
                onClick = { viewModel.loadMovies(forceRefresh
= true) },
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp),
                enabled = movieState !is
MovieViewModel.MovieState.Loading
            ) {
                Text(stringResource(R.string.refresh_movies))
            }
        }

        if (username.isNotEmpty()) {
            Text(
                text = stringResource(R.string.greeting_user,
username),
                style = MaterialTheme.typography.titleMedium,
                modifier = Modifier.padding(16.dp)
            )
        }

        when (val state = movieState) {
            is MovieViewModel.MovieState.Loading -> {
                if (movies.isEmpty()) {
                    LoadingView()
                } else {
                    MovieListView(movies, navController,
isLocal, scrollState, viewModel)
                    LoadingView(modifier =
Modifier.fillMaxWidth())
                }
            }

            is MovieViewModel.MovieState.Error -> {
                ErrorView(state.message)
                if (state.hasData) {
                    MovieListView(movies, navController,
isLocal, scrollState, viewModel)
                } else {
                    EmptyView(isLocal)
                }
            }

```

```
            is MovieViewModel.MovieState.Success -> {
                if (state.isRefreshed) {
                    LaunchedEffect(Unit) {
                        scrollState.scrollToItem(0)
                    }
                }

                if (movies.isEmpty()) {
                    EmptyView(isLocal)
                } else {
                    MovieListView(movies, navController,
isLocal, scrollState, viewModel)
                }
            }
        }
    }
}

@Composable
private fun MovieListView(
    movies: List<Movie>,
    navController: NavController,
    isLocal: Boolean,
    scrollState: LazyListState,
    viewModel: MovieViewModel
) {
    LazyColumn(state = scrollState) {
        items(movies) { movie ->
            MovieCard(
                movie = movie,
                onDetailClick = {
                    if (isLocal) {

viewModel.saveLastViewedMovieId(movie.id)
                    }

navController.navigate("movie_detail/${movie.id}")
                },
                onSaveClick = if (!isLocal) {
                    { viewModel.saveMovieToLocal(movie) }
                } else null
            )
        }
    }
}

@Composable
private fun LoadingView(modifier: Modifier = Modifier) {
    Box(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
```

```
151         contentAlignment = Alignment.Center
152     ) {
153         Column(horizontalAlignment =
154 Alignment.CenterHorizontally) {
155             CircularProgressIndicator()
156             Spacer(modifier = Modifier.height(8.dp))
157             Text(stringResource(R.string.loading_movies))
158         }
159     }
160 }
161
162 @Composable
163 private fun ErrorView(message: String) {
164     Box(
165         modifier = Modifier
166             .fillMaxSize()
167             .padding(16.dp),
168         contentAlignment = Alignment.Center
169     ) {
170         Text(
171             text = stringResource(R.string.error_prefix,
172 message),
173             color = MaterialTheme.colorScheme.error
174         )
175     }
    }

    @Composable
    private fun EmptyView(isLocal: Boolean) {
        Box(
            modifier = Modifier.fillMaxSize(),
            contentAlignment = Alignment.Center
        ) {
            Text(
                text = stringResource(
                    if (isLocal) R.string.no_local_movies else
    R.string.no_remote_movies
                ),
                modifier = Modifier.padding(16.dp)
            )
        }
    }
```

SettingsScreen.kt :

Tabel 13. Source Code Jawaban Soal 1 Jetpack Compose

```
1   package com.example.listcompose.ui.screens
2
3   import androidx.compose.foundation.layout.*
4   import androidx.compose.material3.*
5   import androidx.compose.runtime.*
6   import androidx.compose.ui.Alignment
7   import androidx.compose.ui.Modifier
8   import androidx.compose.ui.res.stringResource
9   import androidx.compose.ui.unit.dp
10  import com.example.listcompose.R
11  import com.example.listcompose.viewmodel.MovieViewModel
12
13  @Composable
14  fun SettingsScreen(
15      viewModel: MovieViewModel,
16      onBackClick: () -> Unit
17  ) {
18      val currentUsername by remember { derivedStateOf {
19  viewModel.getUsername() } }
20      var usernameInput by remember {
21  mutableStateOf(currentUsername) }
22
23      Column(
24          modifier = Modifier
25              .fillMaxSize()
26              .padding(16.dp),
27          horizontalAlignment = Alignment.CenterHorizontally
28      ) {
29          Text(
30              text = stringResource(R.string.settings_title),
31              style = MaterialTheme.typography.headlineMedium,
32              modifier = Modifier.padding(bottom = 24.dp)
33          )
34
35          OutlinedTextField(
36              value = usernameInput,
37              onValueChange = { usernameInput = it },
38              label = {
39  Text(stringResource(R.string.username_label)) },
40              modifier = Modifier.fillMaxWidth()
41          )
42
43          Spacer(modifier = Modifier.height(16.dp))
44
45          Button(
46              onClick = {
47                  viewModel.updateUsername(usernameInput)
```

```
48                  onBackClick()
49              },
50              modifier = Modifier.fillMaxWidth()
51          ) {
52              Text(stringResource(R.string.save_username_button))
53          }
54
55          Spacer(modifier = Modifier.height(8.dp))
56
57          OutlinedButton(
58              onClick = onBackClick,
59              modifier = Modifier.fillMaxWidth()
60          ) {
61              Text(stringResource(R.string.back_button))
          }
      }
  }
```

viewmodel :

MovieViewModel.kt :

Tabel 14. Source Code Jawaban Soal 1 Jetpack Compose

```
1   package com.example.listcompose.viewmodel
2
3   import android.app.Application
4   import androidx.lifecycle.AndroidViewModel
5   import androidx.lifecycle.viewModelScope
6   import com.example.listcompose.data.model.Movie
7   import
8   com.example.listcompose.data.preferences.PreferencesManager
9   import com.example.listcompose.data.remote.MovieRepository
10  import kotlinx.coroutines.flow.MutableStateFlow
11  import kotlinx.coroutines.flow.SharingStarted
12  import kotlinx.coroutines.flow.StateFlow
13  import kotlinx.coroutines.flow.first
14  import kotlinx.coroutines.flow.stateIn
15  import kotlinx.coroutines.launch
16  import timber.log.Timber
17
18  class MovieViewModel(
19      application: Application,
20      private val repository: MovieRepository,
21      private val preferencesManager: PreferencesManager
22  ) : AndroidViewModel(application) {
23
24      sealed class MovieState {
25          data object Loading : MovieState()
26          data class Success(val movies: List<Movie>, val
```

```
isRefreshed: Boolean) : MovieState()
        data class Error(val message: String, val hasData:
Boolean) : MovieState()
    }

    private val _movieState =
MutableStateFlow<MovieState>(MovieState.Loading)
    val movieState: StateFlow<MovieState> = _movieState

    val movieList: StateFlow<List<Movie>> =
repository.getMovies()
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = emptyList()
        )

    val localMovies: StateFlow<List<Movie>> =
repository.getLocalMovies()
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = emptyList()
        )

    fun loadMovies(forceRefresh: Boolean = false) {
        viewModelScope.launch {
            _movieState.value = MovieState.Loading
            try {
                val isRefreshed =
repository.refreshMovies(forceRefresh)
                val currentMovies =
repository.getMovies().first()
                _movieState.value =
MovieState.Success(currentMovies, isRefreshed)
            } catch (e: Exception) {
                val hasData =
repository.getMovies().first().isNotEmpty()
                _movieState.value = MovieState.Error(
                    "Failed to load movies: ${e.message ?:
"Unknown error"}",
                    hasData
                )
                Timber.e(e, "Error loading movies")
            }
        }
    }

    fun saveLastViewedMovieId(id: Int) {
        viewModelScope.launch {
            preferencesManager.saveLastViewedMovieId(id)
        }
```

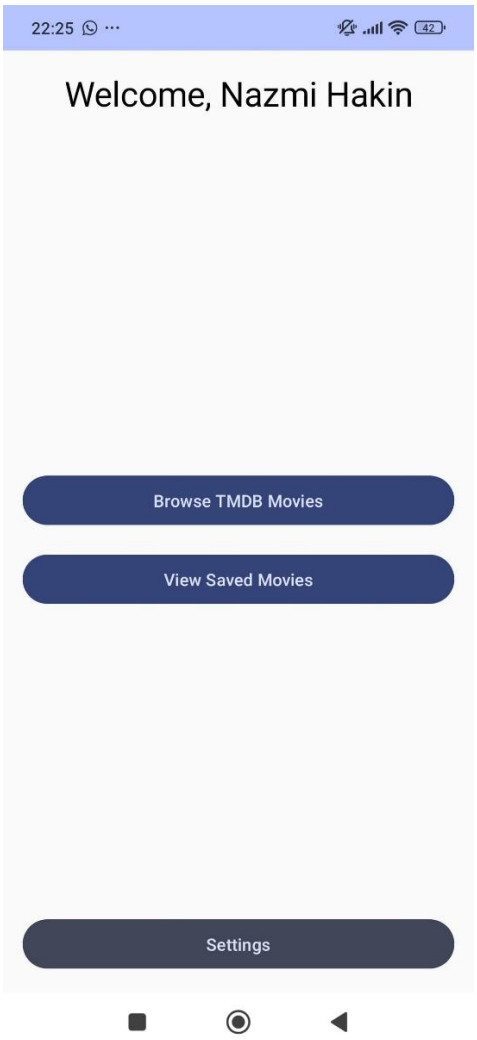
```
79      }
80
81      private fun getLastViewedMovieId(): Int {
82          return preferencesManager.getLastViewedMovieId()
83      }
84
85      fun getLastViewedMovie(): Movie? {
86          val lastId = getLastViewedMovieId()
87          if (lastId == -1) return null
88
89          return findMovieById(lastId)?.takeIf {
90              movieList.value.any { it.id == lastId } ||
91  localMovies.value.any { it.id == lastId }
92          }
93      }
94
95      fun getUsername(): String {
96          return preferencesManager.getUsername()
97      }
98
99      fun updateUsername(newUsername: String) {
100         viewModelScope.launch {
101             preferencesManager.saveUsername(newUsername)
102         }
103     }
104
105     fun saveMovieToLocal(movie: Movie) {
106         viewModelScope.launch {
107             repository.saveMovieToLocal(movie)
        }
    }

    suspend fun getMovieById(id: Int): Movie? {
        return repository.getMovieById(id) ?:
findMovieById(id)
    }

    private fun findMovieById(id: Int): Movie? {
        return localMovies.value.find { it.id == id }
            ?: movieList.value.find { it.id == id }
    }
}
```

MovieViewModelFactory :

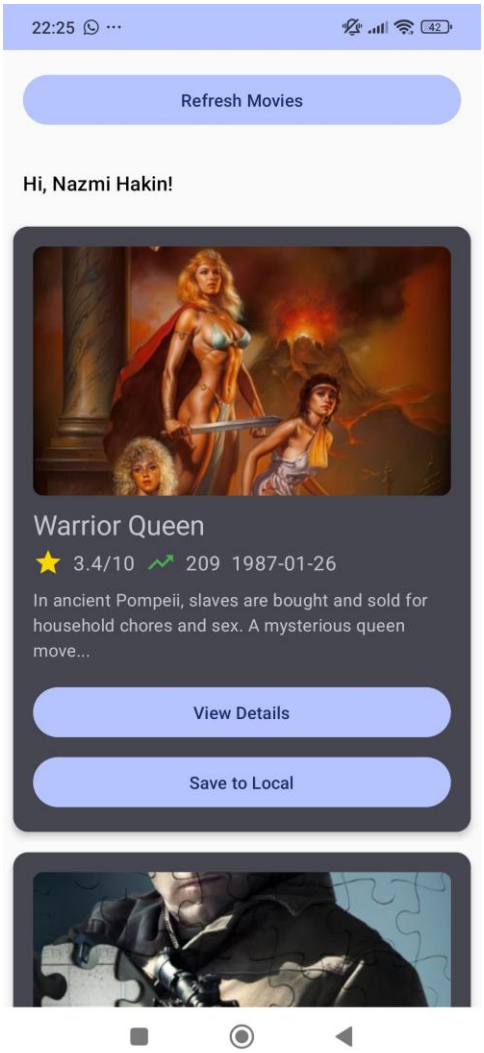
Tabel 15. Source Code Jawaban Soal 1 Jetpack Compose

```
1  package com.example.listcompose.viewmodel
2
3  import android.app.Application
4  import androidx.lifecycle.ViewModel
5  import androidx.lifecycle.ViewModelProvider
6  import
7  com.example.listcompose.data.preferences.PreferencesManager
8  import com.example.listcompose.data.remote.MovieRepository
9
10 class MovieViewModelFactory(
11     private val application: Application,
12     private val repository: MovieRepository,
13     private val preferencesManager: PreferencesManager
14 ) : ViewModelProvider.Factory {
15     override fun <T : ViewModel> create(modelClass: Class<T>):
16 T {
17         if
18 (modelClass.isAssignableFrom(MovieViewModel::class.java)) {
19             @Suppress("UNCHECKED_CAST")
20             return MovieViewModel(application, repository,
21 preferencesManager) as T
           }
           throw IllegalArgumentException("Unknown ViewModel
   class")
     }
   }
```

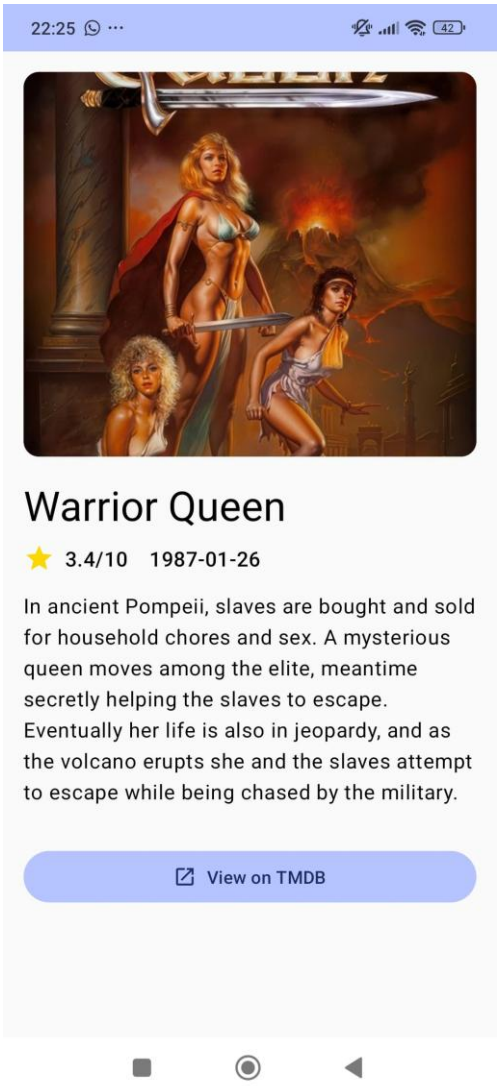
B. Output Program



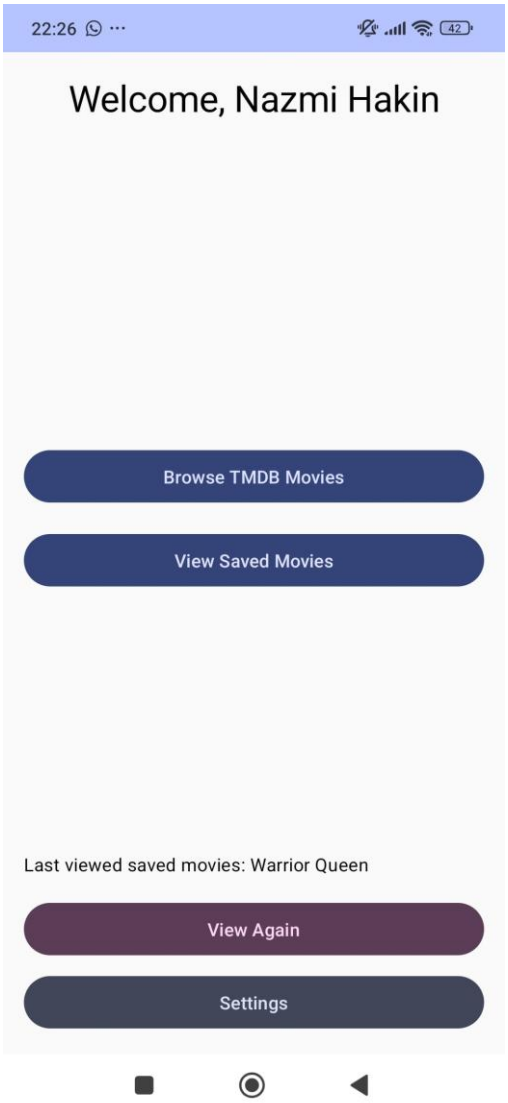
Gambar 1. Screenshot Hasil Jawaban Soal 1 Jetpack Compose



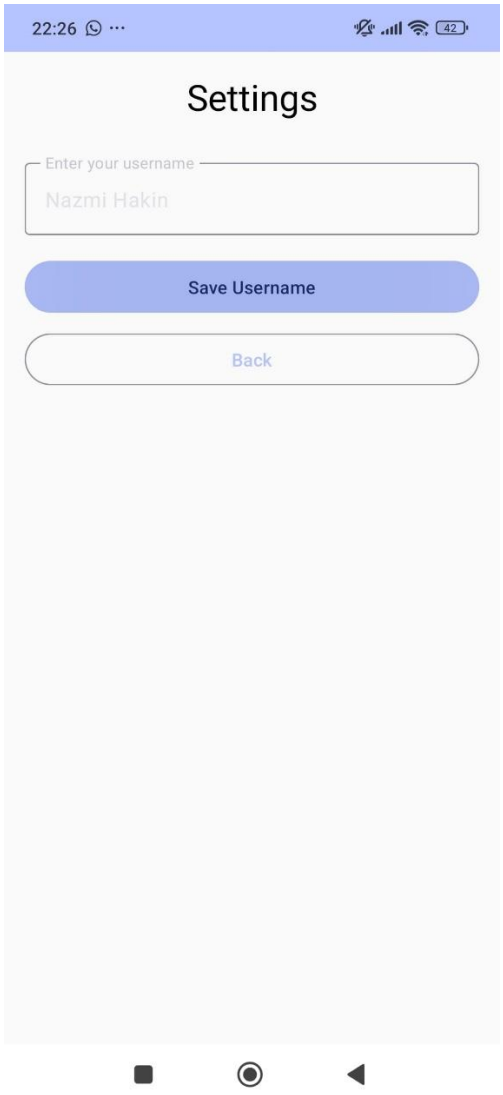
Gambar 2. Screenshot Hasil Jawaban Soal 1 Jetpack Compose



Gambar 3. Screenshot Hasil Jawaban Soal 1 Jetpack Compose



Gambar 4. Screenshot Hasil Jawaban Soal 1 Jetpack Compose



Gambar 5. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

C. Pembahasan

A. Networking dengan Retrofit + Status/Error Handling + Flow :

1. Networking with Retrofit (Pemanggilan API)

TmdbApiService

Ini adalah interface Retrofit yang mendefinisikan endpoint:

```
@GET("movie/popular")
suspend fun getPopularMovies(
    @Query("api_key") apiKey: String,
```

```
    @Query("language") language: String = "en-US",
    @Query("page") page: Int = 1
): MovieResponse
```

- Menggunakan anotasi @GET untuk mengambil daftar **movie populer** dari TMDB.
- suspend artinya fungsi ini dipanggil dalam **coroutine**.

RetrofitInstance

Singleton object untuk membangun Retrofit instance.

```
Retrofit.Builder()
    .baseUrl(BASE_URL)
    .client(client) // dengan logging
    .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
    .build()
    .create(TmdbApiService::class.java)
```

- Menggunakan **kotlinx.serialization** sebagai converter JSON.
- Dilengkapi dengan **HttpLoggingInterceptor** untuk debugging.

2. Flow + Repository Layer (Caching dan Stream Data)

MovieRepository

Bridge antara remote API dan local database (Room).

Fungsi utama:

suspend fun refreshMovies(forceRefresh: Boolean = false): Boolean

- Mengecek apakah perlu **refresh** data dari API (berdasarkan waktu dan flag forceRefresh).
- Jika perlu, akan:
    - Memanggil API → apiService.getPopularMovies(...)
    - Mapping data → ke dalam model Movie

- Simpan ke database lewat movieDao.insertMovies(...)

Fungsi pendukung:

fun getMovies(): Flow<List<Movie>>

fun getLocalMovies(): Flow<List<Movie>>

- Mengembalikan **Flow** dari Room DAO
- Jika ada error saat mengambil dari Room, akan log dan emit emptyList() (graceful fallback)

Cek expired:

private suspend fun isCacheExpired(): Boolean

- Cek waktu lastUpdated movie dan dibandingkan dengan konstanta CACHE_EXPIRATION_TIME_MS.

3. ViewModel Layer + State/Error Handling

MovieViewModel

State Management

```
sealed class MovieState {
   object Loading
   data class Success(val movies: List<Movie>, val isRefreshed: Boolean)
   data class Error(val message: String, val hasData: Boolean)
}
```

- Enum-like class untuk menunjukkan status:
  - Loading: sedang ambil data
  - Success: data berhasil didapat
  - Error: ada kesalahan + info apakah masih ada data lokal

Alur utama: loadMovies()

```
fun loadMovies(forceRefresh: Boolean = false) {
   viewModelScope.launch {
      _movieState.value = MovieState.Loading
      try {
```

```
            val isRefreshed = repository.refreshMovies(forceRefresh)
            val currentMovies = repository.getMovies().first()
            _movieState.value = MovieState.Success(currentMovies, isRefreshed)
        } catch (e: Exception) {
            val hasData = repository.getMovies().first().isNotEmpty()
            _movieState.value = MovieState.Error(
                "Failed to load movies: ${e.message ?: "Unknown error"}",
                hasData
            )
            Timber.e(e, "Error loading movies")
        }
    }
}
```

- Saat dipanggil:
    - Set state ke Loading
    - Coba refresh data dari API
    - Jika sukses → ambil dari database, lalu set Success
    - Jika gagal → ambil data lokal dan set Error dengan hasData = true/false

Live Movie List

```
val movieList: StateFlow<List<Movie>> = repository.getMovies().stateIn(...)
```

- Data dari Room dibungkus jadi StateFlow agar bisa di-observe real-time di Compose UI.

Alur Lengkap: Dari ViewModel → API/DB → UI

1. **UI (Compose)** memanggil viewModel.loadMovies()
2. ViewModel set MovieState.Loading
3. Repository memutuskan:
    - Jika cache valid: ambil dari DB
    - Jika perlu refresh: ambil dari API, simpan ke DB

4. Setelah sukses, ViewModel set MovieState.Success
5. Jika error:
    - Tetap load dari DB jika ada
    - Set MovieState.Error
6. **UI** mereaksi pada perubahan movieState dan movieList via collectAsState() di Compose

B. KotlinX Serialization

1. Penyiapan Konverter KotlinX Serialization

Pada RetrofitInstance.kt:

```
val json = Json {
    ignoreUnknownKeys = true
    isLenient = true
    coerceInputValues = true
}
```

Penjelasan:

- ignoreUnknownKeys = true: jika respons JSON mengandung field yang **tidak didefinisikan** di model Kotlin, akan **diabaikan** (tidak error).
- isLenient = true: memperbolehkan format JSON yang tidak sepenuhnya strict.
- coerceInputValues = true: jika ada field yang nilainya tidak valid, maka akan diganti dengan nilai default.

```
.addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
```

Ini **menghubungkan KotlinX Serialization** sebagai **converter JSON** ke Retrofit.

2. Model Data: Movie dan MovieResponse

@Serializable

```
@Serializable
data class Movie(...)
```

Menandakan bahwa data class Movie dan MovieResponse **bisa diserialisasi dan deserialisasi** oleh KotlinX Serialization.

Contoh dari API TMDB (disingkat):

json

SalinEdit

```
{
  "page": 1,
  "results": [
    {
      "id": 123,
      "title": "Example",
      "overview": "Some description",
      "poster_path": "/abc.jpg",
      ...
    }
  ],
  "total_pages": 100,
  "total_results": 2000
}
```

Akan diubah menjadi:

```
MovieResponse(
    page = 1,
    results = listOf(
        Movie(
            id = 123,
            title = "Example",
            overview = "Some description",
            posterPath = "/abc.jpg",
            ...
```

```
        )
    ),
    totalPages = 100,
    totalResults = 2000
)
```

3. Mapping Nama JSON ke Properti Kotlin

@SerialName("poster_path")

@ColumnInfo(name = "poster_path")

val posterPath: String?,

- @SerialName("poster_path") → memberitahu KotlinX Serialization bahwa JSON poster_path harus dipetakan ke properti Kotlin posterPath.
- Ini **penting karena JSON menggunakan snake_case**, sedangkan Kotlin menggunakan camelCase.

4. Integrasi dengan Room

@Entity(tableName = "movies")

data class Movie(...)

- Karena Movie juga akan disimpan di **Room Database**, maka digunakan juga:
    - @Entity → supaya bisa jadi tabel
    - @ColumnInfo → jika nama kolom Room ingin diubah

Artinya **1 data class Movie digunakan untuk 3 hal sekaligus**:

1. Parsing dari JSON (via KotlinX Serialization)
2. Penyimpanan di Room
3. Penggunaan logika di ViewModel/UI

5. Alur Deserialisasi Sebenarnya

Saat MovieRepository.refreshMovies() dipanggil:

val response = apiService.getPopularMovies(...)

- Retrofit menerima response berupa JSON dari API.
- Converter dari RetrofitInstance akan mem-parsing JSON ke objek MovieResponse, yang di dalamnya ada List<Movie>.
- Semua itu dilakukan secara otomatis oleh KotlinX Serialization **berdasarkan struktur dan anotasi @Serializable**.

Dengan KotlinX Serialization:

- JSON dari TMDB langsung diubah menjadi MovieResponse dan List<Movie> otomatis
- Nama-nama field JSON seperti poster_path, vote_average, dll. bisa langsung dipetakan ke field Kotlin posterPath, voteAverage menggunakan @SerialName
- Tidak perlu membuat DTO (data transfer object) tambahan
- Parsing lebih cepat dan lebih ringan daripada Gson

C. Image Loading dengan Coil

1. Memanggil Coil dengan rememberAsyncImagePainter

```
painter = rememberAsyncImagePainter(
   model = "https://image.tmdb.org/t/p/w500${movie.posterPath}"
)
```

- Fungsi rememberAsyncImagePainter() adalah **pintu masuk utama untuk memuat gambar secara asinkron di Jetpack Compose menggunakan Coil.**
- Parameter model diisi dengan URL gambar dari TMDB API.
- Fungsi ini bersifat **@Composable** dan menggunakan **remember** agar tidak terus-menerus reload saat recomposition.

2. Mengambil Gambar dari Internet

- Coil melakukan **permintaan HTTP ke URL** tersebut menggunakan OkHttp (library HTTP yang digunakan Coil).
- Saat memuat gambar:
    - Jika gambar belum ada di cache, Coil akan mengunduhnya dari internet.

- Jika sudah ada di memori/disk cache, Coil akan langsung menampilkannya tanpa unduh ulang.

3. Caching Otomatis

- Coil **secara otomatis menyimpan gambar ke cache memori dan disk**, jadi kamu tidak perlu mengelola caching sendiri.
- Ini membantu efisiensi aplikasi — tidak perlu mengunduh ulang saat gambar ditampilkan kembali.

4. Menampilkan Gambar di UI

```
Image(
    painter = rememberAsyncImagePainter(...),
    contentDescription = movie.title,
    modifier = Modifier
        .fillMaxWidth()
        .height(200.dp)
        .clip(RoundedCornerShape(8.dp)),
    contentScale = ContentScale.Crop
)
```

- Gambar yang dimuat dari rememberAsyncImagePainter ditampilkan dengan Image().
- contentScale = ContentScale.Crop memastikan gambar memenuhi lebar container dan dipotong sesuai ukuran.
- clip(RoundedCornerShape(8.dp)) membuat sudut gambar membulat.

D. Penggunaan API TMDB

1. API Request dengan Retrofit (TmdbApiService)

```
@GET("movie/popular")
suspend fun getPopularMovies(
    @Query("api_key") apiKey: String,
```

```
    @Query("language") language: String = "en-US",
    @Query("page") page: Int = 1
): MovieResponse
```

- **Endpoint:** /movie/popular
- Menggunakan **@GET Retrofit** untuk memanggil data film populer dari TMDB.
- Parameter:
    - api_key: API Key dari TMDB
    - language: default en-US
    - page: untuk pagination

2. Deserialisasi JSON dengan Kotlinx Serialization

Respons TMDB berupa JSON seperti:

```
{
  "page": 1,
  "results": [
    {
      "id": 123,
      "title": "Movie Title",
      "overview": "...",
      "poster_path": "/abc.jpg",
      "vote_average": 8.5,
      "release_date": "2023-01-01",
      "popularity": 123.45,
      ...
    }
  ],
  "total_pages": 500,
  "total_results": 10000
```

}

- Kotlinx Serialization otomatis mengubah JSON ini menjadi objek Kotlin:
    - MovieResponse untuk respons utama
    - Movie untuk data individual
- Ini dimungkinkan karena kamu menandai @Serializable di data class, dan gunakan @SerialName untuk mencocokkan nama field JSON yang berbeda dari nama properti Kotlin.

3. Pengelolaan Data di Repository (MovieRepository)

val response = apiService.getPopularMovies(...)

- Repository mengelola:
    - **Pengambilan data dari API**
    - **Caching ke Room (local database)**
    - **Pagination & validasi cache**
- Data dari response.results diubah menjadi Movie yang cocok untuk Room, lalu:

movieDao.insertMovies(entities)

- Kemudian disimpan ke lokal (Room DB).

4. Room DAO Menyimpan dan Mengambil Data

Room DAO seperti:

@Query("SELECT * FROM movies")

fun getAllMovies(): Flow<List<Movie>>

- Digunakan untuk mengakses cache lokal sebagai Flow, agar bisa dipantau secara reaktif (otomatis update di UI saat data berubah).

5. Flow Data ke ViewModel (MovieViewModel)

ViewModel menyimpan movieList dan localMovies sebagai:

val movieList: StateFlow<List<Movie>>

val localMovies: StateFlow<List<Movie>>

- ViewModel menerima data dari Repository, dan meneruskannya ke UI dengan cara yang bisa di-*observe*.

6. Tampilkan ke UI (Compose)

Contoh di MovieCard dan MovieDetailScreen:

```
Image(
   painter = rememberAsyncImagePainter(
      model = "https://image.tmdb.org/t/p/w500${movie.posterPath}"
   ),
   ...
)
Text(text = movie.title)
Text(text = movie.overview)
```

- Data ditampilkan sebagai UI card atau detail screen.
- Gambar di-load dari TMDB dengan Coil.
- Data lain seperti title, voteAverage, releaseDate langsung di-bind ke komponen UI.

E. Data Persistence

1. SharedPreferences (PreferencesManager)

Tujuan:

Untuk menyimpan **data ringan dan sederhana**, seperti:

- Username terakhir yang login
- ID film terakhir yang dilihat

Alur Penggunaan:

Simpan Data ke SharedPreferences

Contoh: saat pengguna melihat detail film

```
preferencesManager.saveLastViewedMovieId(movieId)
```

- Disimpan dalam file XML lokal di device
- Hanya key-value pair (string, int, boolean, dll)

Ambil Data dari SharedPreferences

Contoh: menampilkan username atau film terakhir

preferencesManager.getUsername()

preferencesManager.getLastViewedMovieId()

- Dipanggil oleh MovieViewModel saat ingin menampilkan data personal pengguna.

2. Room Database (MovieDao + AppDatabase)

Tujuan:

Menyimpan dan mengambil **data film** secara permanen ke dalam SQLite Database.

1. Menyimpan film dari TMDB ke Room

Di dalam MovieRepository:

movieDao.insertMovies(entities)

- Setelah memanggil API TMDB, respons di-*map* jadi Movie dan disimpan ke DB.
- Pakai OnConflictStrategy.REPLACE untuk memperbarui data jika sudah ada.

2. Mengambil semua film dari database

movieDao.getAllMovies(): Flow<List<Movie>>

- Di-*observe* sebagai Flow dalam MovieViewModel, lalu dikirim ke UI.
- Otomatis update saat data berubah (misalnya saat refresh TMDB API).

3. Cek apakah cache kedaluwarsa

movieDao.getLastUpdateTime()

- Digunakan oleh MovieRepository untuk mengecek kapan terakhir data diperbarui.

4. Menyimpan film favorit ke lokal (mode offline)

movieDao.insertMovies(listOf(movie.copy(isLocal = true)))

- User bisa menyimpan film tertentu secara lokal.

5. Mengambil film lokal (favorit)

movieDao.getLocalMovies(): Flow<List<Movie>>

Integrasi ViewModel

ViewModel menyatukan semuanya:

Ambil data dari Room:

val movieList = repository.getMovies().stateIn(...)

Simpan ke Room:

repository.saveMovieToLocal(movie)

Ambil data kecil dari SharedPreferences:

preferencesManager.getUsername()

preferencesManager.getLastViewedMovieId()

F. Caching Strategy pada Room

1. Ambil Data dari Room (cache) terlebih dahulu

movieDao.getAllMovies()

- MovieViewModel menggunakan getMovies() yang me-*return* Flow<List<Movie>>
- Data ini berasal dari Room dan digunakan untuk menampilkan film di UI secara reaktif

2. Cek apakah cache perlu diperbarui

Metode utama:

suspend fun refreshMovies(forceRefresh: Boolean = false): Boolean

Langkah-langkah pengecekan:

val shouldRefresh = forceRefresh || isCacheExpired() || cachedMovies.isEmpty()

- forceRefresh: Refresh paksa (misal: user swipe to refresh)
- isCacheExpired(): Cek apakah waktu simpan data melebihi batas 30 menit
- cachedMovies.isEmpty(): Tidak ada data di DB sama sekali

3. Cek Kedaluwarsa Cache

```
private suspend fun isCacheExpired(): Boolean {
   return movieDao.getLastUpdateTime()?.let { lastUpdated ->
      System.currentTimeMillis() - lastUpdated > CACHE_EXPIRATION_TIME_MS
   } ?: true
}
```

- Jika last_updated setiap film sudah lebih dari 30 menit, maka dianggap kadaluarsa

4. Jika perlu, ambil data baru dari API

```
val response = apiService.getPopularMovies(...)
```

- Data dari API langsung di-*map* ke entitas Movie lokal, lalu disimpan:

```
movieDao.insertMovies(entities)
```

5. Bersihkan data lama jika forceRefresh = true

```
if (forceRefresh) {
   movieDao.clearAll()
}
```

6. Update lastUpdated saat simpan ke DB

```
lastUpdated = System.currentTimeMillis()
```

- Disimpan dalam properti last_updated di entitas Movie
- Ini yang digunakan sebagai indikator umur data

7. Gunakan data cache dari Room sebagai default

Jika API gagal, MovieViewModel masih bisa mengambil dari:

```
repository.getMovies().first()
```

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.
https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul5